Design code:

```
// mux_if.sv
interface mux_if (input logic clk);
  logic a, b, sel, y;

  modport DUT (
    input  clk, a, b, sel,
    output y
  );

  modport TB (
    input  clk, y,
    output a, b, sel
  );
endinterface

// mux.sv
module mux (mux_if.DUT vif);
  always_ff @(posedge vif.clk) begin
    if (vif.sel)
      vif.y <= vif.b;
    else
      vif.y <= vif.a;
  end
endmodule
```

UVM:

```
`include "uvm_macros.svh"
import uvm_pkg::*;

// ---------------- Transaction ----------------
class mux_tx extends uvm_sequence_item;
  rand bit a, b, sel;

  `uvm_object_utils(mux_tx)

  function new(string name="mux_tx");
    super.new(name);
  endfunction
endclass

// ---------------- Sequence ----------------
class mux_seq extends uvm_sequence #(mux_tx);
  `uvm_object_utils(mux_seq)

  task body();
    mux_tx tx;
    repeat (10) begin
      tx = mux_tx::type_id::create("tx");
      assert(tx.randomize());
      start_item(tx);
      finish_item(tx);
```

```
    end
  endtask
endclass

// ---------------- Driver ----------------
class mux_driver extends uvm_driver #(mux_tx);
  virtual mux_if.TB vif;

  `uvm_component_utils(mux_driver)

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual mux_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
  endfunction

  task run_phase(uvm_phase phase);
    mux_tx tx;
    forever begin
      seq_item_port.get_next_item(tx);
      vif.a   <= tx.a;
      vif.b   <= tx.b;
      vif.sel <= tx.sel;
      @(posedge vif.clk);
      seq_item_port.item_done();
    end
  endtask
```

```
endclass

// ---------------- Sequencer ----------------
class mux_sequencer extends uvm_sequencer #(mux_tx);
  `uvm_component_utils(mux_sequencer)
endclass

// ---------------- Monitor ----------------
class mux_monitor extends uvm_monitor;
  virtual mux_if.TB vif;
  uvm_analysis_port #(mux_tx) ap;

  `uvm_component_utils(mux_monitor)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    ap = new("ap", this);
  endfunction

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual mux_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
  endfunction

  task run_phase(uvm_phase phase);
    mux_tx tx;
```

```
    forever begin
      @(posedge vif.clk);
      tx = mux_tx::type_id::create("tx");
      tx.a   = vif.a;
      tx.b   = vif.b;
      tx.sel = vif.sel;
      ap.write(tx);
    end
  endtask
endclass

// ---------------- Scoreboard ----------------
class mux_scoreboard extends uvm_component;
  uvm_analysis_imp #(mux_tx, mux_scoreboard) analysis_export;
  virtual mux_if.TB vif;

  `uvm_component_utils(mux_scoreboard)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    analysis_export = new("analysis_export", this);
  endfunction

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual mux_if.TB)::get(this,"","vif",vif))
      `uvm_fatal("NOVIF","No virtual interface")
```

```
  endfunction

  function void write(mux_tx tx);
    bit expected_y;
    expected_y = tx.sel ? tx.b : tx.a;
    if (vif.y !== expected_y)
      `uvm_error("SCOREBOARD",$sformatf("Mismatch: sel=%0b a=%0b b=%0b y=%0b
expected=%0b",
                                  tx.sel, tx.a, tx.b, vif.y, expected_y))
    else
      `uvm_info("SCOREBOARD",$sformatf("PASS: sel=%0b a=%0b b=%0b y=%0b",
                                 tx.sel, tx.a, tx.b, vif.y),UVM_LOW)
  endfunction
endclass

// ---------------- Agent ----------------
class mux_agent extends uvm_agent;
  mux_driver    drv;
  mux_sequencer seqr;
  mux_monitor   mon;

  `uvm_component_utils(mux_agent)

  function void build_phase(uvm_phase phase);
    drv  = mux_driver   ::type_id::create("drv", this);
    seqr = mux_sequencer::type_id::create("seqr", this);
```

```
    mon  = mux_monitor  ::type_id::create("mon", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.seq_item_port.connect(seqr.seq_item_export);
  endfunction
endclass

// ---------------- Environment ----------------
class mux_env extends uvm_env;
  mux_agent agent;
  mux_scoreboard sb;

  `uvm_component_utils(mux_env)

  function void build_phase(uvm_phase phase);
    agent = mux_agent::type_id::create("agent", this);
    sb    = mux_scoreboard::type_id::create("sb", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    agent.mon.ap.connect(sb.analysis_export);
  endfunction
endclass

// ---------------- Test ----------------
```

```
class mux_test extends uvm_test;
  mux_env env;
  `uvm_component_utils(mux_test)

  function void build_phase(uvm_phase phase);
    env = mux_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    mux_seq seq;
    phase.raise_objection(this);
    seq = mux_seq::type_id::create("seq");
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass

// ---------------- Top ----------------
module top;
  logic clk = 0;
  always #5 clk = ~clk;

  mux_if vif(clk);

  mux dut (vif);
```

```
  initial begin
    uvm_config_db#(virtual mux_if.TB)::set(null,"*","vif",vif);
    run_test("mux_test");
  end
endmodule
```